EEE 320 INTRODUCTION TO INTERNET OF THINGS
FINAL EXAM
PROJECT APPLICATION HOMEWORK (50P)
2026 Spring

Perform an IoT application (controlling over internet from web or Mobile app with Wifi) on an IoT cloud platform (chosen for final) for your IoT board (chosen for App-1,2,3), sensor (chosen for App-1) and wireless communication module (chosen for App-2), in which you determine the scope. A dc motor (Brushed, Step, or Servo) and one RGB (or at least three LEDs) with one buzzer must be used in your circuit.

Purpose of Application: You are free about the purpose of application. Clearly state the intended purpose (i.e, what you are trying to do for each situation) in your video capture and in this document. BUT, at least you have to send the sensor data from your board to the Cloud interface (dashboard) and also control the circuit (e.g, LED/RGB or any element) from the Cloud interface. i.e, **your project must include both control and monitoring tasks!**

Circuit Diagram: You are free to build your circuit for application. Draw your circuit in **Fritzing.**

Restriction: There is no restriction. You can use any IDE and programming language you want.

Homework Submission: Record a video with all the team members for your application. In your video content; explain your program codes line by line, show your program to be compiled successfully, show your program to be uploaded to your board, show your circuit to be run successfully for each case.

The following files need to be uploaded to Teams.

1. This word document by completing the ANSWERS section (DO NOT upload as pdf)
2. Your video file (MUST be talked in English. Do NOT speak in Turkish! Show your face!)
3. Fritzing circuit file
4. Application project folder created by IDE software. Include your source file

Deadline for submission: 7th June, 2026. Note that 10 points from 50 will be deducted for each day of late uploading!. The system will be closed 4 days later than the deadline!

----------ANSWERS----------

Project Team: Salih Gündüz - Mustafa Burak Başar

Your Board: ESP32-WROOM-32 **(**DOIT ESP32 devkit V1 in Arduino IDE**)**

Your Cloud Platform: Arduino Cloud

Your Sensor: HC-SR04 Ultrasonic Distance Sensor

Wireless Communication Module: Wireless Infrared Remote Control

DC Motor Selected: Servo Motor

Your LED Type: 3 LEDs (Red,Green,Blue)

Your Software IDE: Arduino IDE

Your Programming Language: Arduino Programming Language

Application Purpose:

In this project, it is aimed to develop an ESP32-based smart garage system. The designed system allows the driver to control the garage barrier both autonomously and manually. In the final stage of the project, Arduino Cloud-based control and monitoring features were also integrated into the system.

The project was developed with a modular approach, where each stage added a new capability to the system. In the previous homeworks, the sensor-based autonomous decision mechanism, infrared (IR) wireless communication infrastructure, and physical actuation capability using a servo motor were successfully implemented. In this final homework, internet-based control and monitoring abilities were added to the system.

When the user approaches the garage entrance with the vehicle, the system can be switched into either autonomous mode or manual mode using the infrared remote controller inside the vehicle. The system basically provides two different operating modes and two different control channels. The user can control the system both with the infrared remote controller and through Arduino Cloud using a web or mobile application.

In autonomous mode, the system makes decisions according to sensor data. If the distance between the vehicle and the sensor is greater than 30 cm, the blue LED turns on. If the distance is between 15 cm and 30 cm, the green LED turns on. If the distance becomes less than 15 cm, the system detects the vehicle, the red LED turns on, the buzzer becomes active, and the servo motor rotates to 90 degrees to lift the garage barrier. The system waits for 3 seconds to allow the vehicle to pass. After that, for safety purposes, the barrier slowly returns to its initial position. The servo motor returns to the starting angle, the buzzer turns off, and the red LED turns off.

In manual mode, the user controls the system through the infrared remote controller. The user can control the LEDs and manually open or close the garage barrier. When the red LED case is selected, the buzzer becomes active and the servo motor lifts the barrier. After waiting for 3 seconds, the system slowly returns to its initial position for safety purposes. This manual control structure was optimized compared to the previous homeworks.

With the final homework, Arduino Cloud-based remote control and monitoring features were integrated into the system. The system can now be controlled not only by the infrared remote controller but also through a web or mobile cloud application over the internet. For this feature, both the ESP32 device and the user's control device must be connected to the internet.

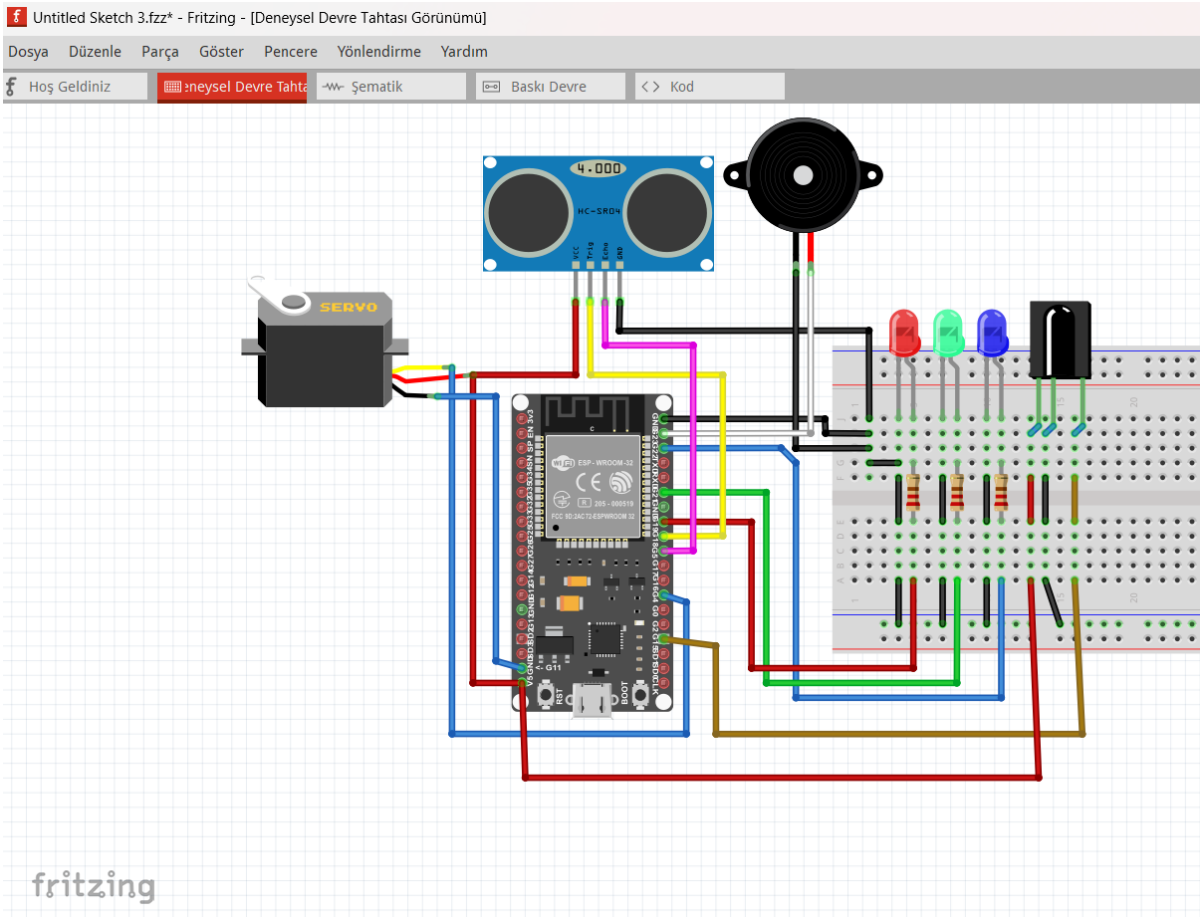
Through the cloud dashboard, the user can change the operating mode, select autonomous or manual mode, monitor sensor data in real time, and observe the distance value numerically. In addition, all manual mode scenarios can also be controlled through the cloud dashboard.

In summary, the system has become a flexible and smart garage system that combines autonomous operation, manual operation, infrared-based control, and cloud-based control together.

With this project, a fully integrated smart garage system capable of autonomous decision making, wireless communication, physical actuation, internet-based control, and real-time monitoring has been successfully developed.

In addition, a 3D model was designed using SolidWorks and produced through 3D printing. This process improved both the functionality and the aesthetic appearance of the system.

Fritzing Circuit Diagram:



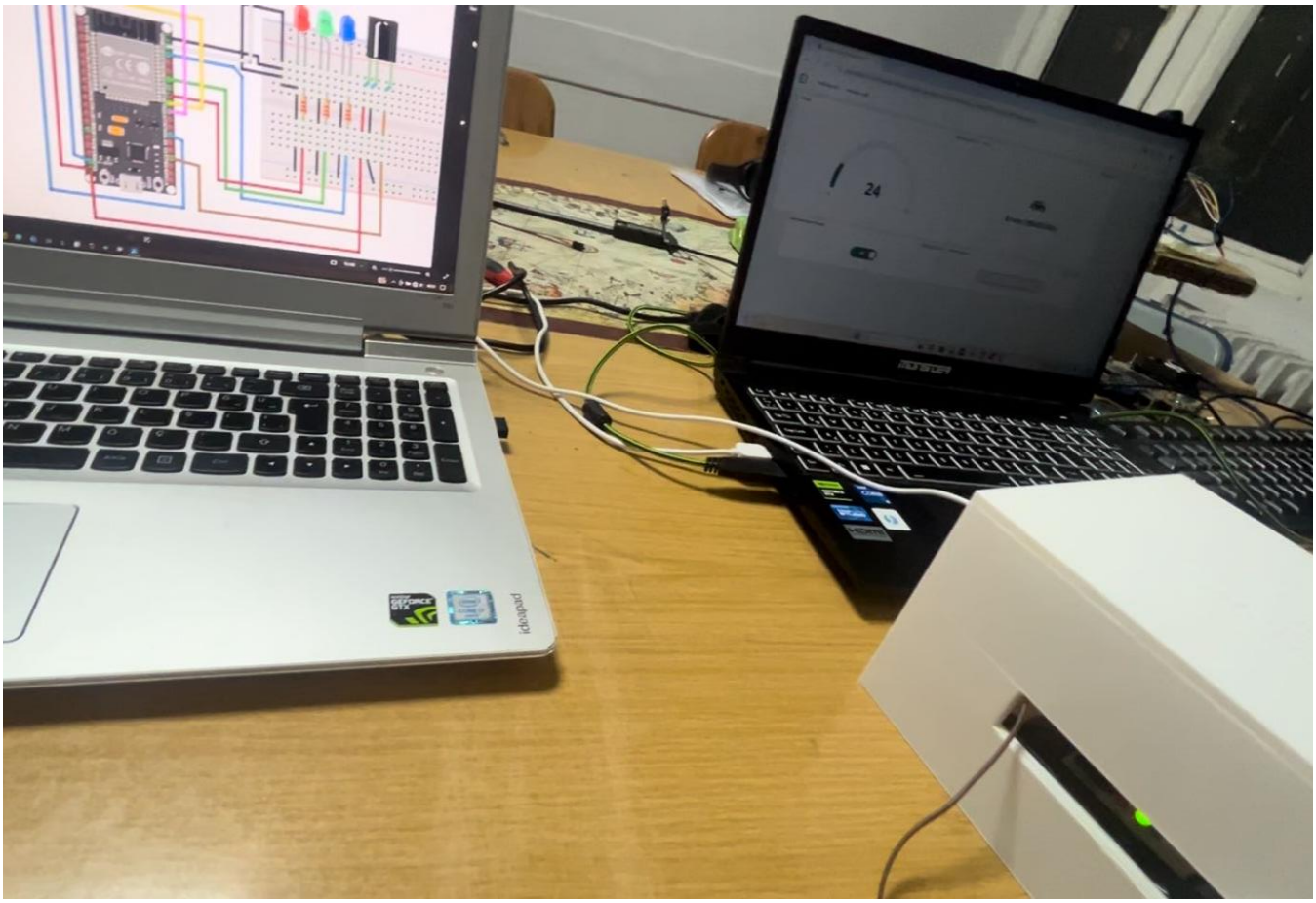
Program codes:

```
#include "thingProperties.h"
#include <ESP32Servo.h>
#include <IRremote.hpp>

// --- PIN DEFINITIONS ---
const int irReceiverPin = 15;
const int trigPin = 5;
const int echoPin = 18;
const int buzzer = 19;
const int ledBlue = 21;
const int ledGreen = 22;
const int ledRed = 23;
const int servoPin = 13;

Servo myServo;
int servoPos = 0;

// --- SERVO AND HARDWARE FUNCTIONS ---
void moveServoUp() {
  for (int pos = servoPos; pos <= 90; pos++) {
    myServo.write(pos);
    ArduinoCloud.update(); // Keep cloud connection alive during servo movement
    delay(10);
  }
  servoPos = 90;
}

void moveServoDownSlow() {
  unsigned long startMillis = millis();
  // Non-blocking wait loop instead of delay(3000)
  while (millis() - startMillis < 3000) {
    ArduinoCloud.update();
  }
  for (int pos = servoPos; pos >= 0; pos--) {
    myServo.write(pos);
    ArduinoCloud.update(); // Keep cloud connection alive during servo movement
    delay(15);
  }
  servoPos = 0;
}

void setLeds(bool b, bool g, bool r, bool buz) {
  digitalWrite(ledBlue, b);
  digitalWrite(ledGreen, g);
  digitalWrite(ledRed, r);
  digitalWrite(buzzer, buz);
  buzzerStatus = buz; // Update buzzer status on the cloud
}

// --- SETUP FUNCTION ---
void setup() {
  Serial.begin(115200);
  delay(1500);

  // Pin Modes
  pinMode(ledRed, OUTPUT);
  pinMode(ledGreen, OUTPUT);
  pinMode(ledBlue, OUTPUT);
  pinMode(buzzer, OUTPUT);
  pinMode(trigPin, OUTPUT);
  pinMode(echoPin, INPUT);

  myServo.attach(servoPin);
  myServo.write(0);

  // Initialize IR Receiver
  IrReceiver.begin(irReceiverPin, ENABLE_LED_FEEDBACK);

  // Initialize Arduino Cloud
  initProperties();
  ArduinoCloud.begin(ArduinoIoTPreferredConnection);
  setDebugMessageLevel(2);
  ArduinoCloud.printDebugInfo();
}
```

```
// --- MAIN LOOP ---
void loop() {
  ArduinoCloud.update(); // The lifeline for cloud communication

  // 1. LISTENING TO IR REMOTE
  if (IrReceiver.decode()) {
    uint16_t command = IrReceiver.decodedIRData.command;

    if (!(IrReceiver.decodedIRData.flags & IRDATA_FLAGS_IS_REPEAT)) {
      switch (command) {
        case 70: // AUTONOMOUS MODE (Remote button 70)
          isAutonomous = true;            // Change cloud variable
          onIsAutonomousChange();         // Update system and dashboard
          break;

        case 64: // MANUAL MODE (Remote button 64)
          isAutonomous = false;           // Change cloud variable
          onIsAutonomousChange();         // Update system and dashboard
          break;

        case 7: // BLUE LED (Remote button 7)
          if (!isAutonomous) {
            // Toggle logic: If on, turn off. If off, turn Blue.
            manualLedSelection = (manualLedSelection == 1) ? 0 : 1;
            onManualLedSelectionChange();
          }
          break;

        case 21: // GREEN LED (Remote button 21)
          if (!isAutonomous) {
            manualLedSelection = (manualLedSelection == 2) ? 0 : 2;
            onManualLedSelectionChange();
          }
          break;
        case 9: // RED LED + SERVO + BUZZER (Remote button 9)
          if (!isAutonomous) {
            manualLedSelection = (manualLedSelection == 3) ? 0 : 3;
            onManualLedSelectionChange();
          }
          break;
      }
    }
    IrReceiver.resume();
  }

  // 2. AUTONOMOUS MODE SENSOR READING
  if (isAutonomous) {
    long duration;
    digitalWrite(trigPin, LOW);
    delayMicroseconds(2);
    digitalWrite(trigPin, HIGH);
    delayMicroseconds(10);
    digitalWrite(trigPin, LOW);

    duration = pulseIn(echoPin, HIGH, 30000);

    if (duration > 0) {
      distance = duration * 0.034 / 2; // Calculate distance and send to cloud

      if (distance > 30) {
        setLeds(HIGH, LOW, LOW, LOW);
        ledStatusInfo = "Blue (Safe)";
      }
      else if (distance >= 15 && distance <= 30) {
        setLeds(LOW, HIGH, LOW, LOW);
        ledStatusInfo = "Green (WARNING)";
      }
      else if (distance < 15) {
        setLeds(LOW, LOW, HIGH, HIGH);
        ledStatusInfo = "Red (STOP!)";
```

```
            ArduinoCloud.update(); // Send data to cloud the moment obstacle is detected
            moveServoUp();
            moveServoDownSlow();
          }
        }
        delay(50);
      }
    }

    // --- CLOUD CALLBACK FUNCTIONS ---

    // Triggered when Mode is changed from Dashboard (or Remote)
    void onIsAutonomousChange()  {
      if (!isAutonomous) {
        setLeds(LOW, LOW, LOW, LOW);
        ledStatusInfo = "Manual Mode Active";
        manualLedSelection = 0; // Reset manual selection when mode changes
      } else {
        ledStatusInfo = "Autonomous Mode Active";
      }
    }

    // Triggered when LED color is selected from Dashboard (or Remote)
    void onManualLedSelectionChange()  {
      if (!isAutonomous) { // Execute only in manual mode
        switch (manualLedSelection) {
          case 0:
            setLeds(LOW, LOW, LOW, LOW);
            ledStatusInfo = "Off";
            break;
          case 1:
            setLeds(HIGH, LOW, LOW, LOW);
            ledStatusInfo = "Blue (Manual)";
            break;
          case 2:
            setLeds(LOW, HIGH, LOW, LOW);
            ledStatusInfo = "Green (Manual)";
            break;
         case 3:
          setLeds(LOW, LOW, HIGH, HIGH);
          ledStatusInfo = "Red (Manual)";
          ArduinoCloud.update(); // Send data to dashboard before servo moves

          moveServoUp();
          moveServoDownSlow();

          // --- ADDED FIX ---
          // Automatically silence buzzer and turn off LED after servo reaches initial position
          setLeds(LOW, LOW, LOW, LOW);
          ledStatusInfo = "Off";
          manualLedSelection = 0; // Reset cloud variable to sync dashboard status
          break;
      }
    }
    }

    // Note: No callback functions (on...Change) are needed for ledStatusInfo,
    // distance, and buzzerStatus since they are set as "Read Only".
```

Photo for your circuit (only 1 photo):



Note: Our system was switched to autonomous mode through Arduino Cloud control. Using Arduino Cloud monitoring, the distance value measured by the sensor in autonomous mode can be observed.

This value is 24 cm, and since it is within the range of 15 < distance < 30 cm, the green LED turns on. This proves that our system successfully performs the correct case operation with Arduino Cloud controlling and monitoring.